TAZROUT

Smart Irrigation System

IoT Field Sensor Node — ESP32 Module

Week 3 Technical Report

Valve Control & Spec-Compliant JSON Packets

Developer	KHENFRI Moussa
Module	ESP32 Field Sensor Node
Firmware	v3.0.0
Spec Reference	ESP32 Technical Specification v1.0
Project	TAZROUT IoT Agricultural Irrigation
Date	March 2026
Status	**Complete — Ready for LoRa integration**

Communication chain: AI Engine → MQTT → Gateway → LoRa → ESP32

Contents

1. Week 3 Objective

The objective of Week 3 was to produce firmware that correctly implements the **TAZROUT ESP32 Technical Specification v1.0**. The previous firmware (v2.0) contained a critical architectural error: the ESP32 was making autonomous irrigation decisions. The specification is explicit on this point.

> **What ESP32 Does NOT Do:**
> "No AI Decisions: ESP32 does NOT make irrigation decisions. It only executes commands received."
> "No WiFi/MQTT: ESP32 does NOT connect to WiFi or MQTT directly."

The three concrete deliverables produced this week:

- `TAZROUT_Node_v1.ino` — Production firmware, spec-compliant, hardware-ready
- `TAZROUT_Node_v2_SIM.ino` — Wokwi simulation version for demonstration
- `diagram.json` — Updated Wokwi diagram (water level sensor added)

2. Critical Architecture Correction

2.1 The 4-Stage Pipeline

The ESP32 is Stage 1 in a four-stage architecture. It only speaks LoRa radio. All irrigation intelligence lives elsewhere.

#	Stage	Role
1	**ESP32 Node**	Reads sensors, controls relay, speaks LoRa only
2	Gateway (RPi + HAT)	Receives LoRa radio, bridges to IP network
3	Gateway Software	Python MQTT client — translates LoRa ↔ MQTT
4	MQTT Broker	Mosquitto/EMQX — routes to Backend, AI, Dashboard

> **Key principle:** The AI Engine makes the irrigation decision. It publishes a `VALVE_COMMAND` to MQTT. The Gateway forwards it via LoRa. The ESP32 receives it and *executes*. The ESP32 has no opinion on whether irrigation is needed.

3. Hardware Configuration

3.1 GPIO Pin Assignment (Spec §3.2)

Signal	GPIO	Type	Notes
DHT22 (temp + humidity)	16	Digital	10kΩ pull-up to 3.3V
Soil Moisture (capacitive)	35	Analog	ADC1_CH7 — inverted output
Water Level (analog)	34	Analog	ADC1_CH6 — **NEW in v3**
Valve Relay (active HIGH)	26	Digital	5V relay, optocoupler isolated
Status LED (built-in)	2	Digital	Heartbeat / *future: LoRa DIO0*
LED DRY (red)	13	Digital	Moisture $< 600\,g/m^3$
LED OPTIMAL (yellow)	12	Digital	Moisture 600–1400 g/m^3
LED WET (green)	14	Digital	Moisture $> 1400\,g/m^3$ / *future: LoRa RST*

GPIO conflict for Week 4: GPIO 2 (heartbeat LED) will be needed for LoRa DIO0, and GPIO 14 (WET LED) for LoRa RST. These LED pins must be reassigned before the SX1276 module is wired.

3.2 ADC Configuration

The ESP32 ADC is configured at the driver level for correct full-range operation:

Listing 1: ADC hardware-level configuration

```
adc1_config_width(ADC_WIDTH_BIT_12);
// GPIO35 --- soil moisture (capacitive, inverted)
adc1_config_channel_atten(ADC1_CHANNEL_7, ADC_ATTEN_DB_12);
// GPIO34 --- water level
adc1_config_channel_atten(ADC1_CHANNEL_6, ADC_ATTEN_DB_12);
```

The 11 dB attenuation (ADC_ATTEN_DB_12) enables the full 0–3.3V input range. Without this, the ADC clips at approximately 1 V.

4. JSON Packet Structures

All five packet types from the specification according to last meeting are fully implemented. The transport layer (`transmitPacket` / `receivePacket`) is abstracted so that replacing Serial with LoRa in Week 4 requires *no changes* to packet logic.

4.1 Packet 1 — SENSOR_READING)

Emitted every 30 seconds. Contains all four sensor readings with status validation per .

```
{
  "packet_type": "SENSOR_READING",
  "zone_id":     "zone_a",
  "zone_name":   "Zone A",
  "device_id":   "MCU-ZONE-A-001",
  "uptime_ms":   30000,
  "sensors": {
```

```
    "temperature":  { "value": "24.5", "unit": "C",    "status":
       "OK" },
    "soil_moisture":{ "value": "540.0","unit": "g/m3", "status":
       "OK" },
    "water_level":  { "value": "62.5", "unit": "%",    "status":
       "OK" },
    "humidity":     { "value": "67.2", "unit": "%",    "status":
       "OK" }
  },
  "signal": { "rssi": 0, "snr": 0, "spreading_factor": 7 }
}
```

Listing 2: SENSOR_READING packet

Note on `signal`: The ESP32 has no knowledge of its own LoRa signal quality. The `rssi` and `snr` fields are set to 0 and overwritten by the Gateway after reception

Sensor status values

Status	Meaning	Example Condition
`OK`	Reading valid and in range	Normal operation
`OUT_OF_RANGE`	Value outside physical bounds	Temp > 80°C
`SENSOR_FAULT`	Sensor not responding / stuck	DHT NaN, ADC at rail

4.2 Packet 2 — COMMAND_ACK

Emitted immediately after receiving and executing (or rejecting) a valve command.

```
{
  "packet_type":       "COMMAND_ACK",
  "command_id":        "CMD-2026-0112",
  "zone_id":           "zone_a",
  "device_id":         "MCU-ZONE-A-001",
  "uptime_ms":         31200,
  "status":            "EXECUTED",
  "valve_state_after": "OPEN",
  "message":           "Valve opened successfully. Timer: 20 min."
}
```

Listing 3: COMMAND_ACK packet — success case

Status values: `EXECUTED` | `FAILED` | `REJECTED` | `COMPLETED`

4.3 Packet 3 — DEVICE_STATE_CHANGE

Emitted on boot. Gateway publishes as a retained MQTT message so the system always knows the last known state.

```
{
  "packet_type":          "DEVICE_STATE_CHANGE",
  "zone_id":              "zone_a",
```

```
  "device_id":              "MCU-ZONE-A-001",
  "uptime_ms":              500,
  "previous_device_state": "OFFLINE",
  "current_device_state":  "ONLINE",
  "valve_state":            "CLOSED",
  "firmware_version":       "3.0.0",
  "battery_level":          -1
}
```

Listing 4: DEVICE_STATE_CHANGE packet — boot

`battery_level: -1` indicates USB-powered prototype (N/A). Will be updated when solar + battery is added in the field deployment phase.

4.4 Packet 4 — EMERGENCY_ALERT

Emitted whenever a sensor crosses a critical threshold. Not dependent on the 30-second cycle — fired immediately after a reading that triggers a threshold.

```
{
  "packet_type":         "EMERGENCY_ALERT",
  "alert_id":            "ALERT-00003",
  "uptime_ms":           90000,
  "severity":            "CRITICAL",
  "issued_by":           "MCU-ZONE-A-001",
  "zone_id":             "zone_a",
  "triggered_by":        "SENSOR_THRESHOLD",
  "sensor_values": {
    "temperature":   58.1,
    "soil_moisture": 120.0,
    "water_level":   4.0,
    "humidity":      21.0
  },
  "message":             "Critical temperature spike and water level
     dangerously low.",
  "recommended_action": "EMERGENCY_IRRIGATION"
}
```

Listing 5: EMERGENCY_ALERT packet

Alert thresholds :

Condition	Threshold	Recommended Action
Temperature too high	$> 45°C$	EMERGENCY_IRRIGATION
Temperature too low	$< 5°C$	STOP_IRRIGATION
Soil moisture low	$< 150\,g/m^3$	EMERGENCY_IRRIGATION
Water level critically low	$< 10\%$	REFILL_TANK
Humidity critically low	$< 20\%$	EMERGENCY_IRRIGATION

4.5 Packet 5 — VALVE_COMMAND Received

The one packet the ESP32 *receives*. Validated for zone and device ID before execution.

```
{
  "packet_type":        "VALVE_COMMAND",
  "command_id":         "CMD-2026-0112",
  "issued_by":          "AI_ENGINE",
  "zone_id":            "zone_a",
  "device_id":          "MCU-ZONE-A-001",
  "command":            "OPEN_VALVE",
  "duration_minutes":   20,
  "priority":           "NORMAL",
  "linked_decision_id":"DEC-2026-0047",
  "reason":             "AI irrigation decision - moisture below
     threshold"
}
```

Listing 6: VALVE_COMMAND packet received from Gateway

5. Valve Control Logic

5.1 Command Execution Flow

1. Receive `VALVE_COMMAND` via LoRa (or Serial in Week 3)
2. Validate `zone_id` and `device_id` match this node — silently discard if not
3. Validate command is `OPEN_VALVE` or `CLOSE_VALVE`
4. **Safety check:** reject if `water_level < 5%`
5. Execute: `GPIO 26 HIGH` (open) or `GPIO 26 LOW` (close)
6. Cap `duration_minutes` at 60 regardless of command value
7. If `duration > 0`: start countdown timer
8. Send `COMMAND_ACK` with `status="EXECUTED"`
9. When timer expires: close valve, send `COMMAND_ACK` with `status="COMPLETED"`

5.2 Safety Mechanisms

Safety Rule	Implementation
Water level check	Reject `OPEN_VALVE` if tank $< 5\%$, send `REJECTED` ACK
Maximum duration	`duration_minutes` capped at 60 min regardless of command
Watchdog timer	Force-close at 65 minutes, log as `SAFETY_WATCHDOG_65MIN`
Double-execution	Check current state before acting; ACK if already in requested state
Power loss	Relay NC configuration — valve closes on power loss (hardware)

6. LoRa Transport Layer

6.1 Packet Framing (Spec §7.2)

Every transmitted packet is wrapped in a 4-byte header and a 2-byte CRC-16/XMODEM trailer:

Header (4B)	JSON Payload	CRC (2B)
len_hi, len_lo, type, version	compact JSON string	CRC-16/XMODEM

Packet type bytes:

Byte	Packet Type	Direction
0x01	SENSOR_READING	Up (ESP32 → Gateway)
0x02	COMMAND_ACK	Up (ESP32 → Gateway)
0x03	DEVICE_STATE_CHANGE	Up (ESP32 → Gateway)
0x04	EMERGENCY_ALERT	Up (ESP32 → Gateway)
0x05	VALVE_COMMAND	Down (Gateway → ESP32)

CRC algorithm: CRC-16/XMODEM (polynomial `0x1021`, initial value `0x0000`). Implemented in `crc16()` function.

6.2 Transport Abstraction — Week 3 vs Week 4

The two functions `transmitPacket()` and `receivePacket()` are the *only* interface between application logic and the radio. No other code knows whether the transport is Serial or LoRa.

Listing 7: Week 4 swap — only these function bodies change

```
// WEEK 3 (current): Serial placeholder
void transmitPacket(uint8_t pktType, const char* json) {
    Serial.println(json);  // prints to Serial Monitor
}

// WEEK 4: Replace body with LoRa calls
void transmitPacket(uint8_t pktType, const char* json) {
    uint16_t len = strlen(json);
    uint16_t crc = crc16((uint8_t*)json, len);
    LoRa.beginPacket();
    LoRa.write((uint8_t)(len >> 8));  LoRa.write((uint8_t)(len & 0xFF));
    LoRa.write(pktType);  LoRa.write(PROTOCOL_VERSION);
    LoRa.write((uint8_t*)json, len);
    LoRa.write((uint8_t)(crc >> 8));  LoRa.write((uint8_t)(crc & 0xFF));
    LoRa.endPacket();  // add 3-retry wrapper here
}
```

> **Zero application changes in Week 4.** `sendSensorReading()`, `sendCommandAck()`, `executeValveCommand()` — none of these functions are touched when LoRa is added. Only the transport bodies change.

7. Week 4 Plan — LoRa Integration

7.1 What Remains

The application layer is complete. Only the transport layer needs physical hardware.

#	Task	Priority	Status
1	Acquire SX1276 868 MHz module + antenna	Blocker	☐ Week 4
2	Resolve GPIO conflicts (reassign LED pins)	Critical	☐ Week 4
3	Wire SX1276 (SCK=18, MISO=19, MOSI=23, CS=5, RST=14, DIO0=2)	Critical	☐ Week 4
4	Replace `transmitPacket()` body with LoRa calls	Core	☐ Week 4
5	Replace `receivePacket()` body with LoRa calls + CRC check	Core	☐ Week 4
6	Add 3-retry logic, 5 s delay (spec §7.3)	Important	☐ Week 4
7	LoRa parameter configuration (SF7, BW125, CR4/5, 17 dBm)	Important	☐ Week 4
8	Range test: RSSI/SNR at 10m, 50m, 100m, 200m	Validation	☐ Week 4
9	End-to-end test with Gateway team (node → LoRa → Gateway → MQTT)	Final	☐ Week 4

7.2 After Week 4

The ESP32 sensor node module will be complete. Remaining integration belongs to other team members: Gateway Python software, MQTT Broker, AI Engine, and Dashboard.

8. Summary — Week 3 Completion

Deliverable / Feature	Status
Architecture corrected — no autonomous decisions	✓ Done
`SENSOR_READING` packet (spec §5.1)	✓ Done
`COMMAND_ACK` packet (spec §5.2)	✓ Done
`DEVICE_STATE_CHANGE` packet (spec §5.3)	✓ Done
`EMERGENCY_ALERT` packet (spec §5.4)	✓ Done
`VALVE_COMMAND` reception + parsing (spec §6.1)	✓ Done
Water level sensor (GPIO 34)	✓ Done
DHT22 on GPIO 16 (corrected from v2)	✓ Done
CRC-16/XMODEM + packet framing (spec §7.2)	✓ Done
Valve safety mechanisms — reject, timer, watchdog (spec §8.3)	✓ Done
LoRa transport abstracted (swap-ready)	✓ Done
Wokwi simulation + updated diagram (2 potentiometers)	✓ Done
SX1276 LoRa TX/RX (physical radio)	☐ Week 4

TAZROUT Smart Irrigation System | KHENFRI Moussa | March 2026
Firmware v3.0.0 | ESP32 Technical Specification v1.0